

JJBike

1. Objetivo.

Construir um armazém de dados para que o administrador da empresa fictícia JJBike possa compreender seu negócio.

2. Modelagem.

Para criar o modelo da estrutura do banco de dados analítico (OLAP) com base no banco de dados transacional (OLTP), utilizando o modelo Star Schema (modelo mais comum para Business Intelligence), foram feitas as seguintes transformações:

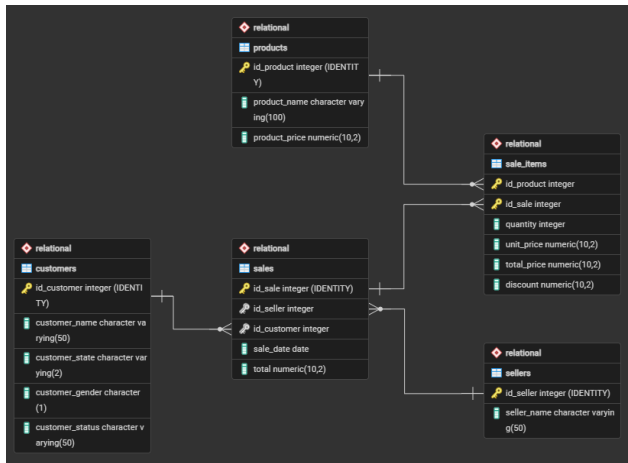
1. Como o objetivo é descrever o modelo de negócio, todas as tabelas do ambiente OLTP que possuem relações um para muitos (1:N) com a tabela *sales* (vendas), foram transformadas em dimensões:

- **Produto ('o quê?'):** *dim_product* (dimensão produto), que possui os atributos da tabela *products* (produtos) do ambiente OLTP (exceto *product_price* (preço do produto), pois isso fica na tabela fato), mais a adição de *validity_start_date* (data de início de validade) e *validity_end_date* (data de fim de validade) para versionamento;
- **Vendedor ('quem?'):** *dim_seller* (dimensão vendedor), que possui os atributos da tabela *sellers* (vendedores) do ambiente OLTP, mais a adição de *validity_start_date* e *validity_end_date* para versionamento;
- **Cliente ('quem?'):** *dim_customer* (dimensão cliente), que possui os atributos da tabela *customers* (clientes) do ambiente OLTP, mais a adição de *validity_start_date* e *validity_end_date* para versionamento.

2. Foi criada a dimensão tempo ('quando?'): *dim_time* (dimensão tempo), que possui o atributo data da tabela *sales* do ambiente transacional, mais os atributos derivados *time_day* (dia), *time_month* (mês), *time_year* (ano), *time_weekday* (dia da semana) e *time_quarter* (trimestre), criados a partir de *time_date* (data), para possibilitar hierarquia de dimensões nas análises;

3. No ambiente OLAP, a tabela fato *fact_sales* (fato vendas) consolidou a tabela de cabeçalho (*sales*) e a de detalhes (*sale_items* (itens de venda) (N:M entre vendas e produtos)), do ambiente OLTP, em um único nível de granularidade: o item de produto por venda. Possuindo sua Surrogate Key (SK) como Primary Key (PK), as SKs das dimensões como suas Foreign Keys (FK) e os atributos da tabela de ligação *sale_items* como métricas.

- A granularidade da tabela fato *fact_sales* foi definida ao nível de item de linha por venda, garantindo o maior nível de detalhamento possível para as análises do administrador da JJBike.



3. pgAdmin 4.

No pgAdmin 4, foi criado o banco de dados 'JJBIKE'.

Obs.: No desenvolvimento do código SQL, será utilizado o padrão de nomenclatura Snake Case.

4. Ambiente Transacional (OLTP) - pgAdmin 4.

1. Foi criado o schema *Relational* (Relacional), que serve como origem para os dados;

2. Foram criadas as tabelas operacionais *sellers*, *products*, *customers*, *sales* e *sale_items* no schema *Relational*.

- A cláusula *GENERATED ALWAYS AS IDENTITY* foi utilizada para gerar automaticamente as Primary Keys das tabelas;
- Todas as colunas, exceto as Primary Keys e *validity_end_date*, foram definidas com a restrição *NOT NULL*, garantindo a integridade dos dados no nível do banco de dados;
- As Restrições de Integridade Referencial *ON DELETE RESTRICT* e *ON DELETE CASCADE* foram utilizadas na tabela da entidade *sale_items*, por ser uma tabela de ligação, garantindo que, respectivamente:
 - Se um registro na tabela *Relational.products* (Relacional.produtos) for tentado ser excluído e estiver sendo referenciado em algum lugar na tabela *sale_items*, o banco de dados emitirá um erro e a exclusão será abortada, evitando que os relatórios de vendas quebrem e dados órfãos sejam criados;
 - Se um registro na tabela *Relational.sales* (Relacional.vendas) for deletado, o banco de dados apagará automaticamente todos os itens relacionados na tabela *sale_items*, pois não faz sentido manter registros órfãos de uma venda excluída.
- Obs.: A restrição padrão ao definir uma FK sem especificar uma ação de exclusão ou atualização é o *NO ACTION*, que, caso se tente deletar um registro na tabela pai que possua dependentes na tabela filha, bloqueia a operação e gera um erro, impedindo a criação de dados órfãos.

3. Foram feitos inserts nas tabelas *sellers*, *products*, *customers*, *sales* e *sale_items*.

Arquivos utilizados (1. Environments/1. OLTP/):

- Pontos 1 e 2: 1. Scripts_Create_Table_Relational.sql;

- **Ponto 3:** 2. *Insert_Into_Customers.sql*, 3. *Insert_Into_Products.sql*, 4. *Insert_Into_Sellers.sql*, 5. *Insert_Into_Sales.sql* e 6. *Insert_Into_SaleItems.sql*.

5. Ambiente Analítico (OLAP) - pgAdmin 4.

5.1. Banco de Dados Analítico (OLAP)

1. Foi criado o schema *Dimensional* (Dimensional), que representa o ambiente analítico (Data Warehouse (DW)) estruturado em um modelo Star Schema;

2. Foram criadas as dimensões *dim_seller*, *dim_customer*, *dim_product*, *dim_time* e a tabela fato *fact_sales* no schema *Dimensional*.

- A cláusula *GENERATED ALWAYS AS IDENTITY* foi utilizada para gerar automaticamente as Surrogate Keys (SK) de todas as dimensões e da tabela fato, servindo como chave primária interna do Data Warehouse (DW);
- Todas as colunas, exceto as Primary Keys, foram definidas com a restrição *NOT NULL*, garantindo a integridade dos dados na camada analítica;
- Foi aplicado o conceito de Slowly Changing Dimensions (SCD) Tipo 2 nas dimensões de vendedores, clientes e produtos por meio das colunas *validity_start_date* e *validity_end_date*, permitindo o versionamento completo e a preservação do histórico dos dados ao longo do tempo.

3. Foram feitos inserts na dimensão tempo. Esse processo foi realizado através de três estruturas principais:

- O bloco *FROM* funciona como um contador, entregando para o restante do código uma tabela temporária com uma coluna chamada *datum* (datum (data)) contendo todas as datas possíveis entre 1970 e 2049;
- O bloco *SELECT* fatia essa data para extrair informações úteis para a análise;
- O bloco *ORDER BY 1* garante que a tabela de dimensão tempo seja inserida em ordem cronológica perfeita, do dia mais antigo para o mais novo.

4. Utilizando CTEs para organizar e simplificar consultas complexas, foram feitas, na ordem descrita, as seguintes cargas de dados do banco de dados transacional (OLTP) para o banco de dados analítico (OLAP):

- **Carga Inicial de clientes, produtos e vendedores:**
 - **CTE S:** Seleciona todos os dados da tabela de origem (ex.: *Relational.customers* (Relacional.clientes)) para servir como base de comparação;
 - **CTE UPD:** Executa um *UPDATE* na tabela de destino (*Dimensional*). A lógica utiliza uma cláusula *WHERE* para identificar registros onde o ID coincide e a *validity_end_date* é nula (registro atual), mas algum atributo (como *seller_name* (nome do vendedor) ou *customer_status* (status do cliente)) mudou. Utiliza *RETURNING* para listar quais registros foram invalidados;
 - **Final INSERT:** Insere novos registros na dimensão. A condição captura tanto os novos registros (que não existem na dimensão) quanto as novas versões dos que acabaram de ser invalidados no bloco *UPD*.
- **Carga da fato vendas (janeiro, fevereiro, março e demais meses):**

- **Lógica de inserção:** O código utiliza um *INSERT INTO ... SELECT* que consolida os dados transacionais;
- **Mapeamento de chaves (SKs):** Realiza *INNER JOIN* entre as tabelas relacionais *sales* e *sale_items* e as tabelas dimensionais, para adquirir os atributos de *sale_items* e as Surrogate Keys das dimensões, respectivamente, e atribuí-los à tabela fato. A regra crucial é o filtro *validity_end_date IS NULL*, que garante que a venda seja associada à versão do cliente/produto que era válida no momento da carga;
- **Filtro temporal:** Utiliza a função *date_part('MONTH', v.sale_date)* para segmentar a carga mês a mês.
- **Atualização de Status (Simulação de SCD Tipo 2):**
 - **Alteração relacional:** Um *UPDATE* simples é feito diretamente na tabela de origem (*Relational.customers*) para simular uma mudança no mundo real;
 - **Processamento de histórico:** Ao rodar o bloco de carga de clientes novamente, o banco detecta que *customer_status* em *S* é diferente do *customer_status* em *dim_customer*. Isso dispara a CTE *UPD* para fechar o registro antigo e criar um novo, preservando o histórico do que o cliente era antes de se tornar 'Gold' ou 'Platinum'.
- **Verificação de Resultados:**
 - **Consultas de Auditoria:** O roteiro finaliza com *SELECT* combinando *fact_sales* com *dim_customer*. Isso prova que vendas antigas continuam ligadas ao status antigo (SK antiga), enquanto vendas novas apontam para o status atualizado (SK nova).

Arquivos utilizados (1. Environments/2. OLAP/):

- **Pontos 1 e 2:** 1. *Scripts_Create_Table_Dimensional.sql*;
- **Ponto 3:** 2. *Insert_Into_dim_time.sql*;
- **Ponto 4:** 3. *Script.sql*.

5.2. Cubo.

O pgAdmin 4 não é um servidor de dados dimensional, portanto foi criada uma tabela desnormalizada (uma única tabela com todos os dados) que une a tabela fato às dimensões para construir o cubo. Esse processo foi realizado através de três estruturas principais:

- O bloco *SELECT* seleciona apenas as colunas de interesse para a análise (excluindo as SKs, colunas de ID e atributos de versionamento), extraíndo os atributos textuais das dimensões e as métricas numéricas da tabela fato;
- O bloco *INTO* define a criação física da nova tabela no banco de dados, especificando *Dimensional.sales_cube* (*Dimensional.cubo_vendas*) como destino final — uma nova tabela no schema dimensional que armazenará os dados unificados;
- O bloco *FROM* conecta a tabela *Dimensional.fact_sales* (*Dimensional.fato_vendas*) a todas as tabelas dimensionais ao redor dela por meio de *INNER JOINs*, utilizando as chaves substitutas (SKs) para consolidar o relacionamento estrela (Star Schema).

Arquivos utilizados (1. Environments/2. OLAP/):

4. *Cube.sql*.

5.3. KPI (Indicador de Performance).

O pgAdmin 4 não é um servidor de dados dimensional, portanto foi criada uma tabela desnormalizada (uma única tabela com todos os dados) para medir a receita total da empresa fictícia JJBike. Esse processo foi realizado através de quatro estruturas principais:

- O bloco *SELECT* seleciona apenas as colunas de interesse para a análise macro, extraindo o atributo temporal (mês), calculando a métrica agregada de receita total (*SUM*) da tabela fato e gerando dinamicamente a coluna de metas por meio de uma estrutura condicional (*CASE*);
- O operador de conversão *::numeric*, posicionado logo após o fechamento do bloco condicional (*END*), atua como um atalho nativo do PostgreSQL para a função *CAST*. Ele força a coluna de metas gerada dinamicamente a se consolidar como um tipo de dado numérico decimal exato, garantindo a padronização matemática e a compatibilidade de formatos com a métrica de receita realizada ao gerar os cálculos de atingimento no Power BI;
- O bloco *INTO* define a criação física da nova tabela no banco de dados, especificando *Dimensional.kpi* (*Dimensional.kpi*) como destino final, que armazenará os dados consolidados de resultados e metas por mês;
- O bloco *FROM* conecta a tabela central *Dimensional.fact_sales* à dimensão *Dimensional.dim_time* (*Dimensional.dim_tempo*) por meio de um *INNER JOIN* utilizando as chaves substitutas (*SKs*), agrupando (*GROUP BY*) e ordenando (*ORDER BY*) os resultados finais por mês.

Arquivos utilizados (1. Environments/2. OLAP/):

5. KPI.sql.

6. Artefatos - Power BI:

1. As tabelas do banco de dados *Dimensional*, criadas no pgAdmin 4, foram conectadas ao Power BI.

6.1. Relatórios.

1. Foram feitas as seguintes alterações nas tabelas:

- Em *fact_sales*, foi criada uma coluna chamada *discount_percent* (percentual de desconto) (com duas casas decimais), com a seguinte fórmula: $discount_percent = TRUNC(('dimensional fact_sales'[discount] / 'dimensional fact_sales'[total_price]) * 100, 2)$;
- Em *dim_customer*, foi criada uma coluna chamada *full_location* (localização completa), com a seguinte fórmula:
 - *location_map* =
SWITCH(
'dimensional dim_customer'[customer_state],
"AC", "Acre", "AL", "Alagoas", "AM", "Amazonas", "AP", "Amapá", "BA",
"Bahia", "CE", "Ceará", "DF", "Distrito Federal", "ES", "Espírito Santo",
"GO", "Goiás", "MA", "Maranhão", "MG", "Minas Gerais", "MS", "Mato
Grosso do Sul", "MT", "Mato Grosso", "PA", "Pará", "PB", "Paraíba", "PE",
"Pernambuco", "PI", "Piauí", "PR", "Paraná", "RJ", "Rio de Janeiro", "RN",
"Rio Grande do Norte", "RO", "Rondônia", "RR", "Roraima", "RS", "Rio

Grande do Sul", "SC", "Santa Catarina", "SE", "Sergipe", "SP", "São Paulo", "TO", "Tocantins", "Unknown"

) & ", Brazil".

- Em *dim_kpi*, foram criadas três métricas para que, no relatório de KPI, o card tenha comportamento dinâmico de acordo com o que estiver selecionado nos botões do button slicer:

- measure_target* =

IF(

ISFILTERED('dimensional kpi'[month]) || *ISFILTERED('dimensional dim_time'[time_year])*,

SUM('dimensional kpi'[target]),

0

).

- measure_total_revenue* =

IF(

ISFILTERED('dimensional kpi'[month]) || *ISFILTERED('dimensional dim_time'[time_year])*,

SUM('dimensional kpi'[total_revenue]),

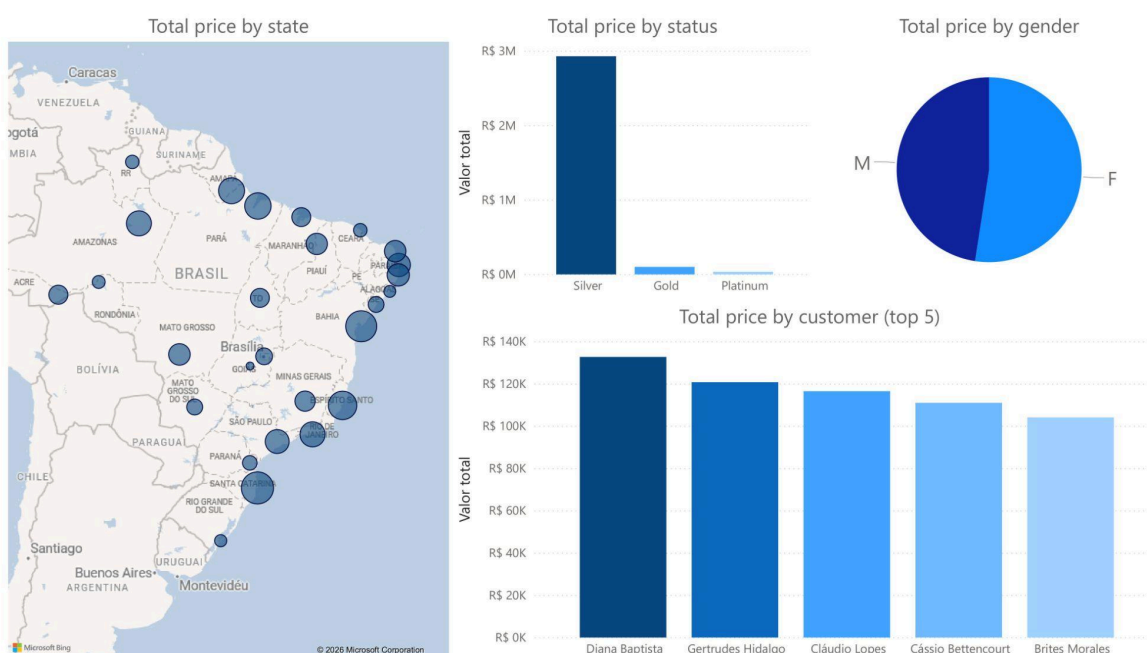
0

).

2. Foram criados os seguintes relatórios:

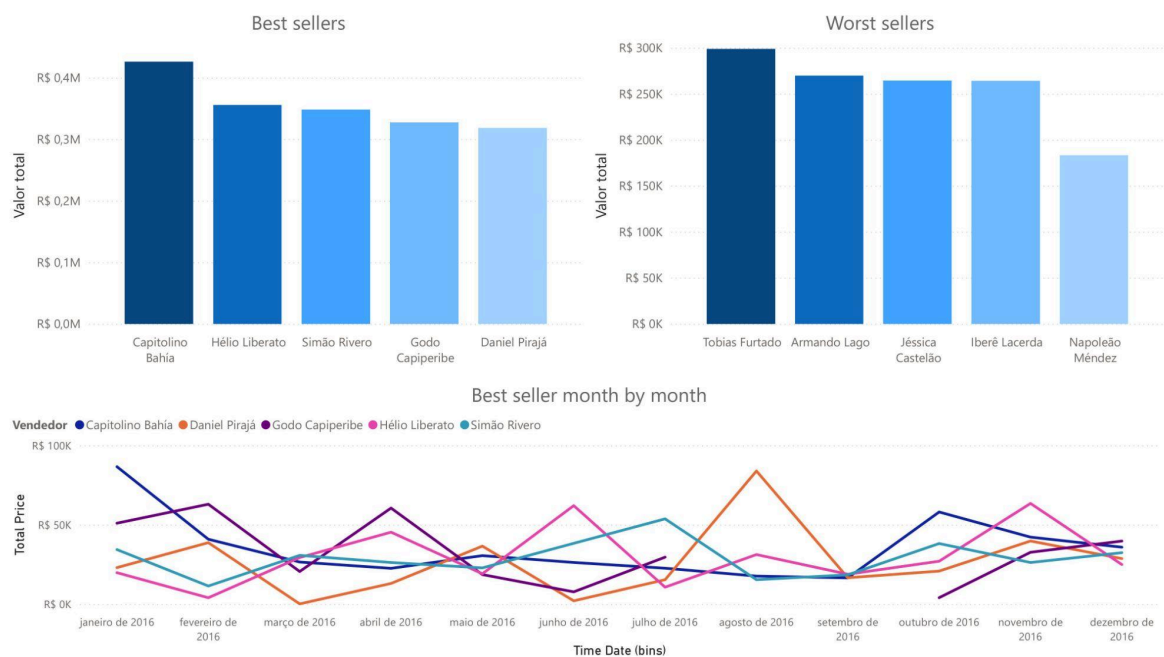
(Clientes)

O relatório de clientes apresenta: somatório de valor total por estado, somatório de valor total por status, somatório de valor total por gênero e somatório de valor total por cliente (top 5).



(Vendedores)

O relatório de vendedores apresenta: somatório de valor total por vendedor (top 5 melhores vendedores, top 5 piores vendedores e top 5 melhores vendedores comparados mês a mês).



(Produtos)

O relatório de produtos apresenta: somatório de valor total por produto (top 5 de produtos mais vendidos e top 5 de produtos menos vendidos) e somatório de desconto por produto (top 5 de produtos com mais desconto acumulado e top 5 de produtos com menos desconto acumulado).



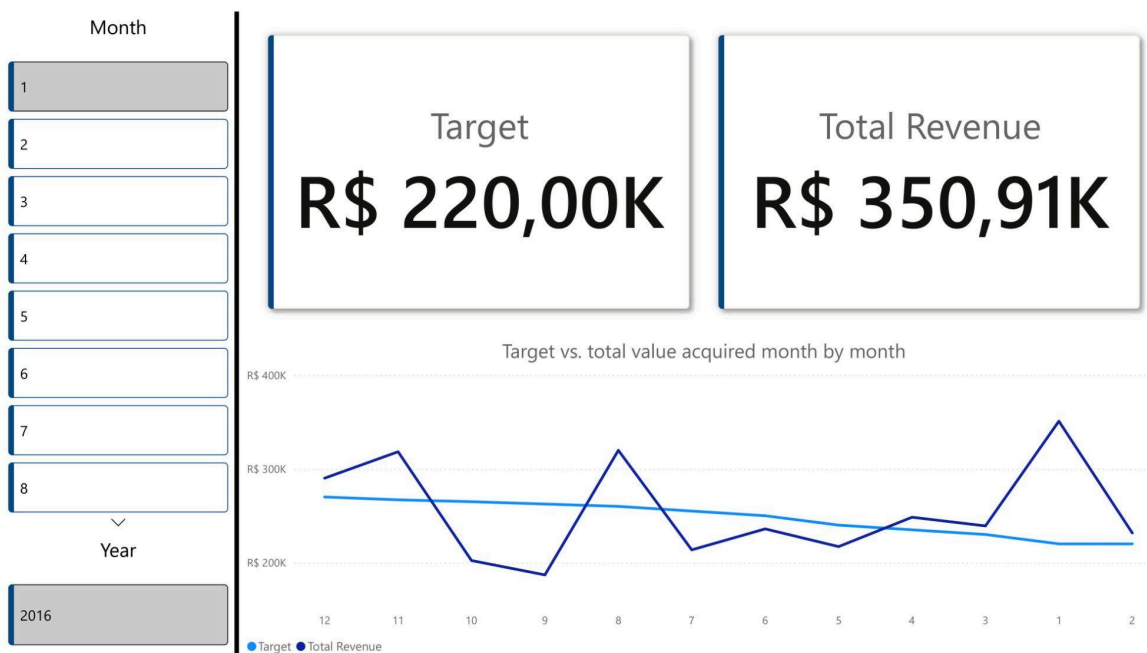
(Vendas)

O relatório de vendas apresenta: somatório de valor total de produtos vendidos (comparados mês a mês), somatório de quantidade de produtos vendidos (comparados mês a mês), somatório de desconto de produtos vendidos (comparados mês a mês) e cards de produtos, vendedores e clientes para filtragem.



(KPI)

O relatório de KPI apresenta: meta, valor total adquirido, somatório de meta e valor total adquirido (comparados mês a mês) e cards de mês e ano para filtragem.



6.2. Cubo.

1. Foram criadas as seguintes hierarquias a partir das dimensões:

- **Geographic Hierarchy** (Hierarquia Geográfica): *customer_state* (estado do cliente) → *customer_name* (nome do cliente) (*dim_customer*);
- **Date Hierarchy** (Hierarquia Temporal): *time_year* → *time_quarter* → *time_month* → *time_day* (*dim_time*);
- Observações:
 - *customer_gender* (gênero do cliente) e *customer_status* (de *dim_customer*) devem ficar livres no painel lateral como Atributos de Dimensão Soltos, pois inseridos na *Geographic Hierarchy* implicaria que são parte de uma subdivisão geográfica;
 - *dim_seller* e *dim_product* possuem apenas um atributo além de suas chaves, portanto não geraram hierarquias e seus atributos tornaram-se Atributos de Dimensão Soltos.

2. Representação visual do cubo criado:

Customer State	Customer Name	Year	Quantity	Discount	Total Price
AC	Alarico Quintero	2016	1		R\$ 155,00
				R\$ 60,45	
	Total			R\$ 155,00	
	Antônio Sobral		3	R\$ 3.334,61	R\$ 16.787,60
	Basilio Soares		18	R\$ 2.516,00	R\$ 42.237,47
	Carla Briones		1	R\$ 587,75	R\$ 2.260,58
	Ester Castanho		5	R\$ 2.327,33	R\$ 15.416,98
	Evaristo Bahia		3	R\$ 692,17	R\$ 4.836,70
	Irene Villanueva		3	R\$ 1.654,34	R\$ 7.183,75
	Total		34	R\$ 11.172,65	R\$ 88.878,08
AL			6	R\$ 4.199,33	R\$ 26.817,61
AM			57	R\$ 15.881,10	R\$ 168.571,45
AP			53	R\$ 16.202,60	R\$ 183.945,80
BA			65	R\$ 18.104,59	R\$ 274.690,93
CE			11	R\$ 9.146,31	R\$ 37.562,17
DF			19	R\$ 14.839,02	R\$ 65.272,18
ES			89	R\$ 12.590,55	R\$ 226.925,84
GO			3	R\$ 858,20	R\$ 7.614,38
MA			38	R\$ 5.149,81	R\$ 87.431,06
MG			31	R\$ 18.853,70	R\$ 102.980,97
MS			19	R\$ 11.094,30	R\$ 57.340,96
MT			44	R\$ 11.279,89	R\$ 119.272,51
PA			82	R\$ 24.476,00	R\$ 190.925,91
PB			51	R\$ 8.791,16	R\$ 143.634,26
PE			42	R\$ 19.477,64	R\$ 129.473,93
PI			44	R\$ 20.851,47	R\$ 116.457,99
PR			22	R\$ 5.322,31	R\$ 45.718,33
RJ			57	R\$ 19.318,44	R\$ 169.913,05
RN			45	R\$ 16.227,19	R\$ 118.559,35

Arquivos gerados disponíveis em '2. Artifacts/'.